

PROJECT DOCUMENTATION REPORT

Library Management System

Student Name	B.Kaviya
Roll No	922525148057
Department	AIML
Project Type	Full-Stack CRUD Web Application
Project Repository	github.com/kaviyabalusamy15-sketch/library_management_system

Overview

The Library Management System is a complete CRUD-based web application designed to manage books, members, and book loan records. The application separates the React frontend from the Django REST Framework backend and uses REST APIs to communicate with a SQLite relational database. It supports adding, viewing, editing and deleting books and members, issuing and returning books, searching books, validation, and responsive user interaction.

1. Problem Statement

Manual library record management can be time-consuming and may lead to duplicate or outdated records. Maintaining books, members, and loan information manually can make searching, updating, and tracking availability difficult. This project addresses these issues through a web-based library management interface with CRUD operations, REST APIs, validation, and persistent database storage.

2. Objectives

- Implement complete CRUD operations.
- Store library records in a relational database.
- Provide REST APIs for books, members, and loans.
- Validate user input on the frontend and backend.
- Demonstrate integration between a React frontend and Django REST Framework backend.
- Support book issuing, returning, and book search functionality.

3. Technology Stack

Layer	Technology	Purpose
Frontend	React 18.3.1 / Vite 5.4.0	Dynamic client-side interface and user interaction
Frontend Styling	Custom CSS	Responsive layouts and presentation
HTTP Client	Axios 1.7.0	Communication with REST API
Backend	Python / Django 5.x	Server-side application and routing
API	Django REST Framework 3.15+	REST API endpoints and CRUD operations
CORS	django-cors-headers 4.4+	Cross-origin communication
Database	SQLite	Persistent relational data storage
API Testing	Postman	REST API inspection and testing
Development	VS Code	Runtime execution and development

4. System Architecture

The application follows a decoupled client-server web architecture:

User Browser (React Frontend) → REST API → Django REST Framework → Django ORM → SQLite Database

Client Layer: The React interface displays books, members, and loan records, accepts form input, performs client-side validation, and sends HTTP requests using Axios.

REST API Layer: Django REST Framework receives requests, validates serialized data, performs CRUD operations, and returns JSON responses.

Database Layer: Django ORM maps the Book, Member, and Loan models to SQLite database tables and provides persistent storage.

5. Database Design and Schema

The main entities managed by the system are Book, Member, and Loan.

Entity 1: Book

- id – Integer primary key and auto-generated identifier.
- title – Book title.
- author – Author name.
- isbn – Unique ISBN value.
- category – Book category.
- published_year – Year of publication.
- quantity – Total number of copies.
- available_quantity – Number of copies currently available.

Entity 2: Member

- id – Integer primary key.
- name – Member name.
- email – Unique email address.
- phone – Member phone number.
- joined_date – Automatically recorded joining date.

Entity 3: Loan

- id – Integer primary key.
- book – Foreign key referencing Book.
- member – Foreign key referencing Member.
- issue_date – Automatically recorded issue date.
- due_date – Date by which the book should be returned.
- return_date – Return date, when applicable.
- status – Loan status such as Issued or Returned.

6. User Interface and Functional Modules

Books Module

The Books section allows the user to add new books, view the current list, edit existing book details, delete records, and search books by title, author, ISBN, or category. Quantity and available quantity are displayed for each record.

Members Module

The Members section provides forms and tables for adding, viewing, editing, and deleting library members. The email field uses email-format validation and member email values are unique.

Loans Module

The Loans section allows an available book to be issued to a selected member. The system creates a due date 14 days from the issue date. A loan can subsequently be marked as Returned, with the return date recorded.

Search and Feedback

The frontend provides book search filtering and displays notification messages after successful or failed operations. When the backend is unavailable, the interface reports that the Django server should be started.

7. REST API Endpoint Documentation

Operation	HTTP Method	Endpoint	Purpose
Read All Books	GET	/api/books/	Retrieves all books
Create Book	POST	/api/books/	Creates a book record
Read One Book	GET	/api/books/{id}/	Retrieves a specific book
Update Book	PUT	/api/books/{id}/	Updates a book
Delete Book	DELETE	/api/books/{id}/	Removes a book
Read All Members	GET	/api/members/	Retrieves members
Create Member	POST	/api/members/	Creates a member
Read/Update/Delete Member	GET/PUT/DELETE	/api/members/{id}/	Manages a specific member
Read/Create/Update/Delete Loan	GET/POST/PUT/PATCH/DELETE	/api/loans/ and /api/loans/{id}/	Manages loan records

8. CRUD Implementation Details

- **Create (POST):** Users enter book or member details in the form. The frontend sends the data to the REST API, and Django saves the record in SQLite.
- **Read (GET):** On application load, Axios requests retrieve books, members, and loans. The returned JSON data is displayed in the React interface.
- **Update (PUT):** Selecting Edit loads the existing record into the form. Submitting the form sends a PUT request to update the backend record.
- **Delete (DELETE):** The user confirms deletion, after which the frontend sends a DELETE request and refreshes the displayed records.
- **Issue:** An available book and member are selected, and a POST request creates a loan with a 14-day due date.
- **Return:** A PATCH request changes the loan status to Returned and records the current return date.

9. Validation

- Required fields are enforced in frontend forms.
- Email format is validated for members.
- Phone input follows a 10–15 digit pattern.
- ISBN values are unique.
- Member email values are unique.
- Available quantity is constrained by the model/application logic.
- Backend validation prevents issuing unavailable books.

10. Testing Results and Verification

Test Case	Tested Action	Expected Result	Status
TC-01	Add valid book	Book saved and displayed	PASSED
TC-02	Duplicate ISBN	Validation/API error shown	PASSED
TC-03	Edit book	Book details updated	PASSED
TC-04	Delete book	Book record removed	PASSED
TC-05	Add valid member	Member saved and displayed	PASSED
TC-06	Duplicate email	Validation/API error shown	PASSED
TC-07	Issue available book	Loan created and availability updated	PASSED
TC-08	Issue unavailable book	Request rejected by backend	PASSED
TC-09	Return issued book	Loan status changed to Returned	PASSED
TC-10	Backend stopped	Frontend shows backend connection message	PASSED

11. Installation and Execution Steps

Prerequisites

- Python 3.10+ installed.
- Node.js 18+ installed.
- Google Chrome, Microsoft Edge, or another modern browser.
- VS Code or another code editor.
- **Backend:** Open a terminal and run `cd backend`.
- Create and activate a virtual environment if required: `python -m venv venv` and on Windows `venv\Scripts\activate`.
- Install dependencies: `pip install -r requirements.txt`.
- Apply migrations: `python manage.py migrate`.
- Start Django: `python manage.py runserver`. The backend runs at `http://127.0.0.1:8000`.
- **Frontend:** Open a second terminal, run `cd frontend`, then `npm install`.
- Start Vite using `npm run dev`. The frontend runs at `http://localhost:5173`.

12. Challenges and Solutions

CORS Communication: The frontend and backend run on different origins during development. CORS support is configured through `django-cors-headers` so that browser requests can reach the Django API.

Real-Time UI Synchronization: After Create, Update, Delete, Issue, or Return operations, the React application reloads the latest records so that changes are visible without manually refreshing the page.

Book Availability: The issue-book interface filters the list to books with available quantity greater than zero, while backend validation also handles unavailable-book cases.

13. Future Enhancements

- Login and role-based access for administrators and members.
- Fine calculation for overdue books.
- Dashboard charts and library statistics.
- Pagination for large book/member/loan datasets.
- Advanced search and filtering.
- PostgreSQL deployment for production use.

14. Git Repository and Reference Details

Project Repository: https://github.com/kaviyabalusamy15-sketch/library_management_system

Version Control: Git and GitHub Web

Project structure contains separate frontend and backend directories, documentation, and a Postman API collection.

Primary project flow: React UI → Django REST API → Django ORM → SQLite database.

Student: B.Kaviya **Roll No:** 922525148057 **Department:** AIML

Thank You